



Smart Textile Manufacturing Quality Inspection and Production Optimization Platform

CODE REVIEW AND REPOSITORY EVALUATION

Submitted in the partial fulfilment for the Course of

CSA1016

Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

INFORMATION TECHNOLOGY

Submitted by

B Priyanka (192421206)

Under the supervision of

Dr. K. Anithadavamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai - 602105

August 2026

1. Problem Overview

1.1 Problem Statement

The Smart Textile Manufacturing Quality Inspection and Production Optimization Platform is designed to support textile manufacturing through digital quality inspection, production monitoring, and machine-performance monitoring. The system aims to reduce dependence on manual inspection and improve the consistency and efficiency of manufacturing operations.

1.2 Implemented Solution

The current project is a prototype developed using FastAPI. It provides REST API functionality, a health-check endpoint, Swagger-based API documentation, Docker containerization, Docker Compose deployment, and Git-based version control.

The prototype provides the foundation for future features such as AI-based fabric defect detection, production monitoring, machine monitoring, reporting, alerts, and production optimization.

1.3 Project Objectives

The main objectives are:

- Improve textile quality inspection.
- Support production monitoring.
- Support machine-performance monitoring.
- Reduce manual inspection effort.
- Provide a maintainable software platform.
- Apply Git for version control.
- Use Docker for consistent deployment.

1.4 Key Functionalities

The current prototype implements:

- FastAPI REST API.
- /health endpoint for application health monitoring.
- Swagger UI for API documentation and testing.
- Docker-based application deployment.
- Docker Compose configuration.
- Git branching and version control.
- README-based project documentation.

Advanced textile inspection, AI-based defect detection, production optimization, and monitoring features are planned as future enhancements.

2. Repository Organization

2.1 Repository Structure

The repository follows a simple structure that separates application source code, dependencies, Docker configuration, version-control configuration, and documentation.

Smart-Textile-Platform/

```
|— app/
|   |— main.py
|— .gitignore
|— Dockerfile
|— docker-compose.yml
|— README.md
|— requirements.txt
```

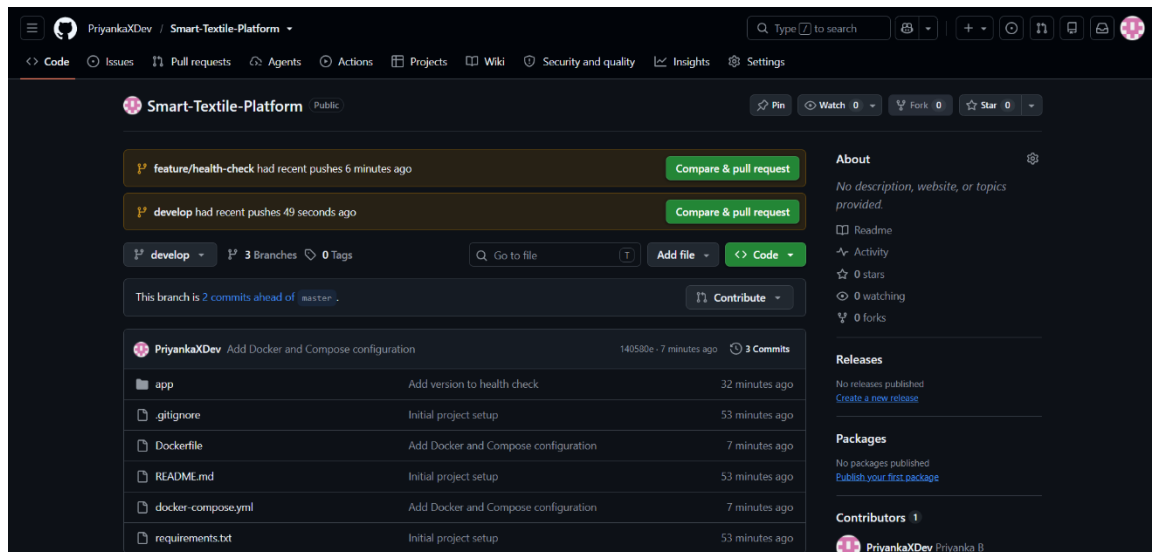


Figure 2.1 –Project Repository Structure

2.2 Folder and File Organization

File/Folder	Purpose
app/	Contains the application source code.
main.py	Contains the FastAPI application and API endpoints.
requirements.txt	Defines the required Python dependencies.
Dockerfile	Defines the Docker image build instructions.
docker-compose.yml	Defines the Docker Compose service and port configuration.
.gitignore	Prevents unnecessary files from being tracked by Git.
README.md	Provides project documentation and usage information.

The structure is simple and easy to navigate, making the repository suitable for maintenance and collaboration.

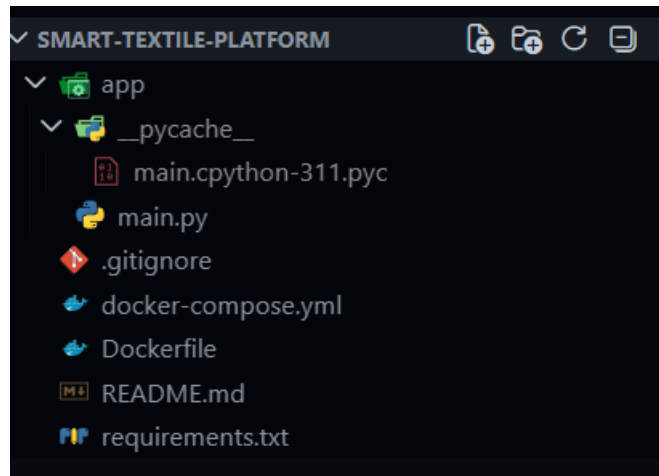


Figure 2.2 – Repository Folder Structure

3. Code Quality Review

3.1 Code Readability

The FastAPI source code is short and easy to understand. The endpoint names such as /health clearly describe their purpose, and the application uses consistent Python formatting.

The main limitation is that most application logic is currently contained in app/main.py. This is acceptable for the prototype but may become difficult to maintain as more features are added.

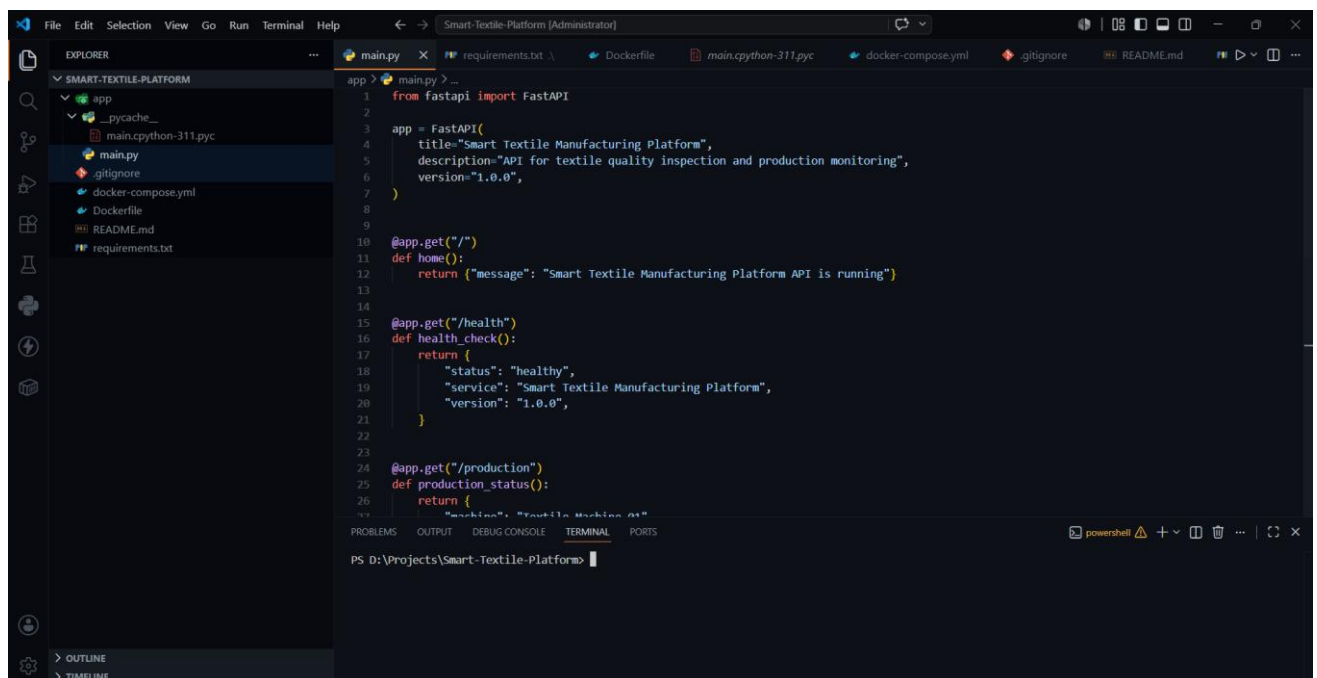


Figure 3.1 – Source Code Readability

3.2 Code Modularity

The repository separates application code from configuration files. However, the current application is mainly implemented in a single main.py file.

For future development, routes, services, models, schemas, and configuration should be separated into dedicated modules. This would improve maintainability and make individual components easier to test.

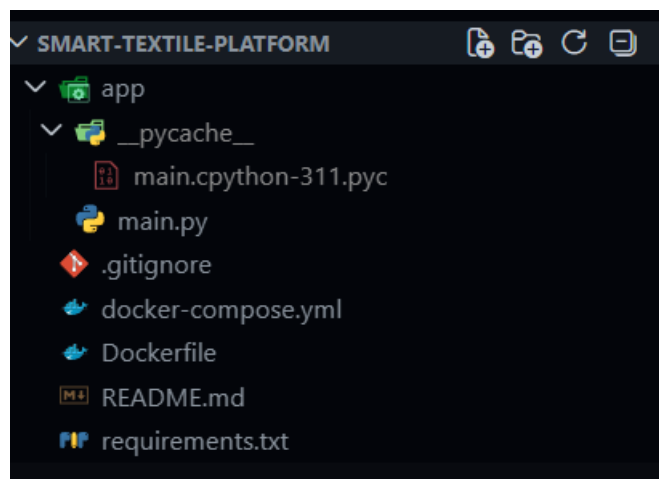


Figure 3.2 – Modular Code Structure

3.3 Coding Standards

The code follows a simple and consistent Python style. Function and endpoint names are understandable, and the FastAPI route structure is clear.

Automated formatting, linting, and static analysis should be introduced as the project grows to maintain consistent coding standards.

3.4 Code Documentation

FastAPI metadata and Swagger UI provide basic API documentation. However, source-code documentation is limited.

Additional docstrings and comments should be added for important functions and complex implementation logic as the application becomes larger.

3.5 Code Quality Strengths

- Simple and readable implementation.
- Clear API endpoint definitions.
- Consistent project structure.
- FastAPI provides automatic API documentation.
- Small codebase is easy to understand.
- Docker provides a consistent execution environment.

3.6 Areas for Improvement

- Divide main.py into separate modules.
- Add structured error handling.
- Add automated unit and integration tests.
- Introduce linting and formatting tools.
- Add source-code documentation.
- Improve scalability through modular architecture.

4. Version Control Evaluation

4.1 Commit History

The repository maintains a clear commit history covering the main stages of project development. The commits describe specific changes, making the development process easier to track and maintain.

```
PS D:\Projects\Smart-Textile-Platform> git add Dockerfile docker-compose.yml
PS D:\Projects\Smart-Textile-Platform> git commit -m "Add Docker and Compose configuration"
[feature/health-check 140580e] Add Docker and Compose configuration
2 files changed, 21 insertions(+)
PS D:\Projects\Smart-Textile-Platform> git log --oneline --decorate -3
140580e (HEAD -> feature/health-check) Add Docker and Compose configuration
1ed67b9 (origin/feature/health-check) Add version to health check
dc19ad0 (origin/master, master) Initial project setup
PS D:\Projects\Smart-Textile-Platform>
```

Figure 4.1 – Git Commit History

4.2 Branching Strategy

The project uses a feature-based branching strategy with master, develop, and feature/health-check branches. The feature branch allows changes to be developed separately before integration into develop.

```
PS D:\Projects\Smart-Textile-Platform> git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
PS D:\Projects\Smart-Textile-Platform> git branch -vv
* develop          140580e [origin/develop] Add Docker and Compose configuration
  feature/health-check 140580e [origin/feature/health-check] Add Docker and Compose configuration
  master           dc19ad0 [origin/master] Initial project setup
PS D:\Projects\Smart-Textile-Platform>
```

Figure 4.2 – Git Branching Strategy

4.3 Commit Messages

The commit messages are short and describe the purpose of each change, such as:

- Initial project setup
- Add version to health check
- Add Docker and Compose configuration
- Add project documentation

Meaningful commit messages improve traceability and make the repository history easier to understand.

4.4 Repository Maintenance

Git is used to track source-code changes, manage branches, integrate features, and synchronize the project with GitHub. The .gitignore file prevents unnecessary files from being committed.

The feature branch was merged into develop, demonstrating the use of Git for controlled feature integration and repository maintenance.

5. Code Testing and Validation

5.1 Test Cases

The application was tested to verify that the FastAPI service and its health-check functionality operate correctly.

Test Case	Expected Result	Actual Result	Status
Start application	Application starts successfully	Application started successfully	Pass
Access /docs	Swagger UI loads	Swagger UI loaded	Pass
Execute /health	HTTP 200 with healthy status	HTTP 200 received	Pass
Check Docker container	Container remains running	Container status shows Up	Pass

5.2 Test Execution

The application was executed using Docker Compose and verified using docker compose ps. The /health endpoint was then tested through Swagger UI.

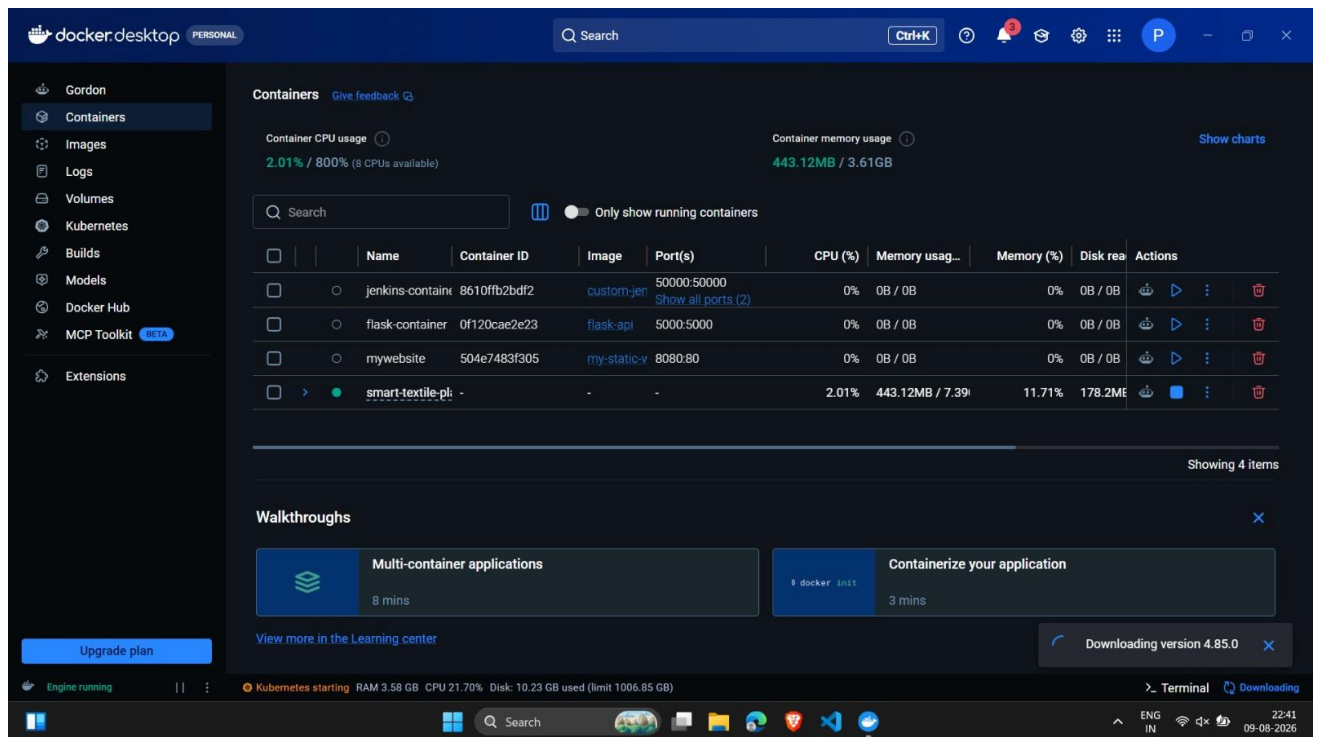


Figure 5.1 – Docker Container Running in Docker Desktop

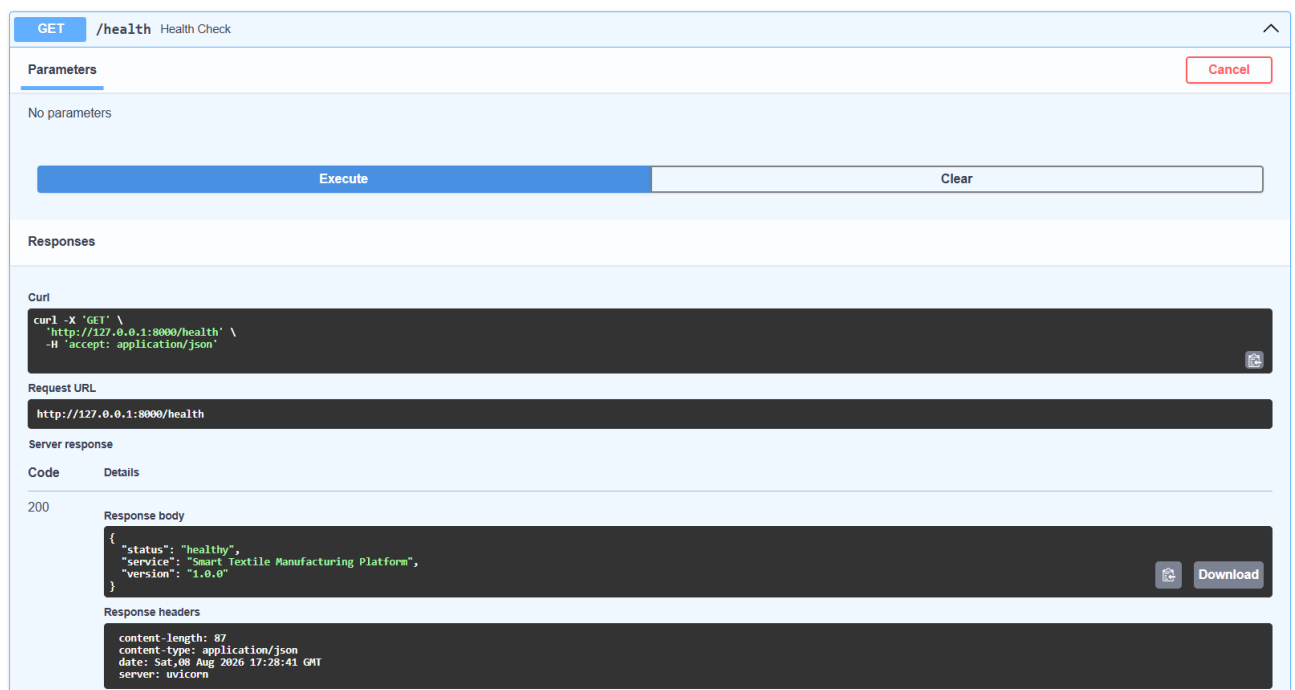


Figure 5.2 –Successful Health Check API Test

```
PS D:\Projects\Smart-Textile-Platform> docker compose logs
smart-textile-compose | INFO:      Started server process [1]
smart-textile-compose | INFO:      Waiting for application startup.
smart-textile-compose | INFO:      Application startup complete.
smart-textile-compose | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
smart-textile-compose | INFO:      172.19.0.1:49344 - "GET /docs HTTP/1.1" 200 OK
smart-textile-compose | INFO:      172.19.0.1:49344 - "GET /openapi.json HTTP/1.1" 200 OK
smart-textile-compose | INFO:      172.19.0.1:53974 - "GET /health HTTP/1.1" 200 OK
PS D:\Projects\Smart-Textile-Platform>
```

Figure 5.3 – Docker Container Execution and Application Logs

5.3 Validation Result

The test results confirm that the application starts successfully inside the Docker container, the Swagger documentation is accessible, and the /health endpoint returns the expected successful response. The Docker logs also confirm successful application startup and API request processing.

6. Repository Documentation

6.1 README Evaluation

The README provides basic information about the Smart Textile Manufacturing Platform, including the project purpose, features, technology stack, project structure, and execution details.

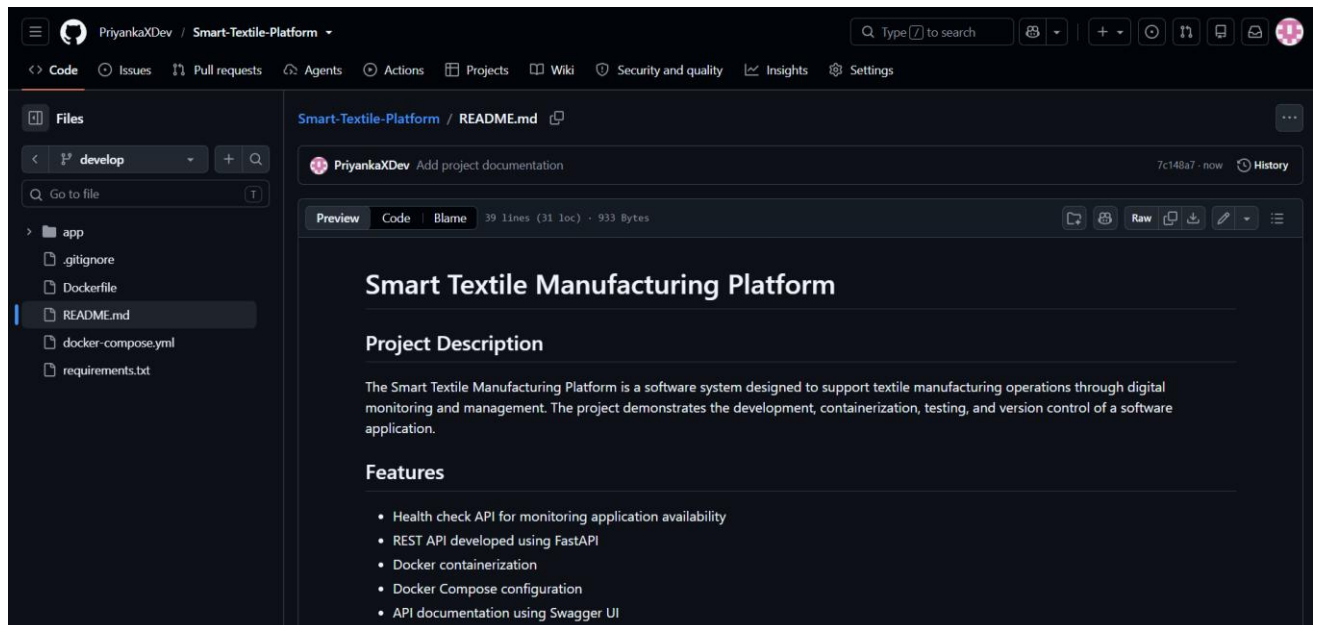


Figure 6.1 – README Documentation

6.2 Installation and Usage

The documentation provides the basic steps required to set up and run the application. Docker and Docker Compose instructions help users build and start the application consistently.

6.3 Dependencies

The project uses requirements.txt to maintain the Python dependencies required by the FastAPI application. This makes dependency installation easier and supports reproducible setup.

6.4 Documentation Improvements

The README can be improved by adding:

- Detailed API endpoint descriptions.
- Example API requests and responses.
- Troubleshooting instructions.
- Testing procedures.
- Development guidelines.
- Deployment instructions for production environments.

These improvements would make the repository easier for new developers to understand, install, test, and maintain.

7. Code Review Findings and Recommendations

7.1 Overall Findings

The code review shows that the project has a simple repository structure and a readable FastAPI implementation. Git, Docker, Docker Compose, README documentation, and Swagger UI are used effectively in the current prototype.

The main limitations are that application logic is concentrated in `main.py`, source-code documentation is limited, and automated testing and code-quality tools are not yet implemented.

7.2 Recommendations

- Separate routes, services, models, and configuration into modules.
- Add unit and integration tests.
- Improve error handling and input validation.
- Add docstrings and source-code documentation.
- Introduce linting and formatting tools.
- Maintain clear and meaningful Git commits.
- Keep the README updated with setup, API, testing, and troubleshooting information.
- Introduce CI/CD for automated testing and deployment.

7.3 Future Enhancements

Future development can include database integration, authentication, AI-based fabric defect detection, production monitoring, machine monitoring, reporting, dashboards, and cloud deployment.

These improvements would increase the project's scalability, maintainability, reliability, and usability.

GitHub Repository:

<https://github.com/PriyankaXDev/Smart-Textile-Platform>